🎯 GLA UNIVERSITY — DAY 2 IN-CLASS SOCRATIC CHECKPOINT MCQS (OFFICIAL INSTRUCTOR KEY)

High Performance Computing & Heterogeneous AI Systems (`GLA-HPC-BCSG701`)

Lead Trainer: Shivam Gupta | **Target Session:** Day 2 Lectures & Labs (Interactive In-Class Checkpoints)
Total Questions: 25 Multi-Choice Questions | **Modules Covered:** 1 through 5 (Day 2 Syllabus)

📌 Pedagogical Delivery Strategy (Socratic In-Class Checkpointing)

Do NOT administer these 25 questions as a single sit-down exam! Instead, weave 5 questions into each of the 5 syllabus topics as you teach. When you finish explaining a concept (e.g., Fork-Join, Reduction, `MPI_Allreduce`, or `patch_sklearn`), project the corresponding question, give students 30 seconds to commit to an answer, and use the explanation to **connect the dots** to the next architectural concept!

Question 01 [Module 1: Parallel Computing Principles | Easy | Foundational Concept]

Topic: Concurrency vs Parallelism
Question: What is the precise architectural difference between 'Concurrency' and 'True Parallelism' in high-performance computer systems?

- **a)** Concurrency executes simultaneously on multiple physical cores; Parallelism interleaves tasks via time-slicing on a single CPU core
- **b)** Concurrency is the composition of independently executing processes managed via interleaving; Parallelism is the simultaneous physical execution of multiple computational tasks at the exact same instant across multiple physical hardware cores
- **c)** Concurrency only applies to Python, while Parallelism only applies to C++
- **d)** Concurrency requires a GPU, while Parallelism requires an FPGA

> 💡 **TIP**
> **Correct Answer: [b]** — Concurrency is the composition of independently executing processes managed via interleaving; Parallelism is the simultaneous physical execution of multiple computational tasks at the exact same instant across multiple physical hardware cores
> **Technical Explanation:** Concurrency is about structure (handling multiple things at once via rapid context switching or time-slicing on one or more cores). Parallelism is about physical execution (doing multiple things at the exact same physical nanosecond on distinct physical CPU/GPU cores).

Question 02 [Module 1: Parallel Computing Principles | Easy | Hardware Architecture]

Topic: Shared Memory vs Distributed Memory Architectures
Question: Which of the following statements accurately characterizes the memory address space in Shared-Memory

(SMP/NUMA) versus Distributed-Memory (Cluster) computing?

- **a)** In Distributed-Memory systems, all CPU cores across different cluster racks can directly dereference the same raw pointer address without networking
- **b)** In Shared-Memory systems, all processors access a globally shared physical address space; in Distributed-Memory systems, each node possesses private local memory requiring explicit message passing over a network interconnect to share data
- **c)** Shared-Memory systems require InfiniBand switches between CPU cores on the same motherboard
- **d)** Distributed-Memory systems cannot execute C++ applications

> **TIP**
> **Correct Answer: [b]** — In Shared-Memory systems, all processors access a globally shared physical address space; in Distributed-Memory systems, each node possesses private local memory requiring explicit message passing over a network interconnect to share data
> **Technical Explanation:** In shared-memory architectures (e.g., dual-socket Xeon), all cores read and write to a unified global address space over memory buses. In distributed-memory clusters, physical memories are physically isolated per node, necessitating explicit communication protocols like MPI across network fabrics.

Question 03 [Module 1: Parallel Computing Principles | Medium | Mathematical Derivation]

Topic: Amdahl's Law Serial Bottleneck
Question: An engineer parallelizes an ML feature extraction pipeline such that 80% is parallelized across 16 CPU cores, while 20% remains strictly sequential. What is the theoretical overall speedup achieved according to Amdahl's Law?

- **a)** 16.0x
- **b)** 4.0x
- **c)** 5.0x
- **d)** 8.0x

> **TIP**
> **Correct Answer: [b]** — 4.0x
> **Technical Explanation:** Amdahl's Law: Speedup = 1 / [S + (P / N)] = 1 / [0.20 + (0.80 / 16)] = 1 / [0.20 + 0.05] = 1 / 0.25 = 4.0x speedup.

Question 04 [Module 1: Parallel Computing Principles | Easy | Parallel Paradigms]

Topic: Task Parallelism vs Data Parallelism
Question: Training a Random Forest where different CPU threads train independent decision trees on different subsets of the data represents which parallel computing paradigm?

- **a)** Instruction-Level Speculation
- **b)** Task Parallelism combined with Data Parallelism
- **c)** Serial Pipelining
- **d)** Lock-step SIMD Vectorization

> **TIP**
> **Correct Answer: [b]** — Task Parallelism combined with Data Parallelism
> **Technical Explanation:** Random Forest builds an ensemble of trees concurrently (Task Parallelism: each thread builds a distinct estimator) operating on distinct bootstrap subsets of the training samples (Data Parallelism: partitioning data across workers).

Question 05 [Module 1: Parallel Computing Principles | Medium | Performance Metrics]

Topic: Parallel Efficiency Metric
Question: If a simulation runs in 100 seconds on 1 core and in 20 seconds on 8 cores, what is the Parallel Efficiency (E) of this scaling?

- **a)** 80%
- **b)** 62.5%
- **c)** 50%
- **d)** 25%

> **TIP**
> **Correct Answer: [b]** — 62.5%
> **Technical Explanation:** Speedup S = T_1 / T_P = 100s / 20s = 5.0x. Parallel Efficiency E = Speedup / P = 5.0 / 8 = 0.625 = 62.5%.

Question 06 [Module 2: OpenMP for Multithreading | Easy | Execution Model]

Topic: OpenMP Fork-Join Execution Model
Question: What happens during the transition from serial to parallel execution in the OpenMP Fork-Join programming model?

- **a)** The computer creates a new virtual machine
- **b)** The single initial Master Thread encounters an OpenMP parallel directive and forks a team of worker threads; at the end of the parallel region, worker threads synchronize at an implicit barrier and join back into the master thread
- **c)** The operating system reboots the CPU socket into hyper-threading mode
- **d)** All threads are terminated and written to disk swap

> **TIP**
> **Correct Answer: [b]** — The single initial Master Thread encounters an OpenMP parallel directive and forks a team of worker threads; at the end of the parallel region, worker threads synchronize at an implicit barrier and join back into the master thread
> **Technical Explanation:** OpenMP relies on the Fork-Join model: execution begins with a single master thread. Upon reaching `#pragma omp parallel`, the master thread forks a team of worker threads. Upon reaching the closing brace, threads synchronize at an implicit barrier and join, leaving only the master thread.

Question 07 [Module 2: OpenMP for Multithreading | Medium | Parallel Syntax & Correctness]

Topic: The Reduction Clause in OpenMP
Question: In `#pragma omp parallel for reduction(+:total_sum)`, how does the OpenMP compiler prevent data races

when accumulating values across millions of loop iterations?

- **a)** It forces all threads to execute one at a time sequentially
- **b)** It allocates a private local copy of `total_sum` initialized to 0 for each thread, accumulates locally without synchronization during the loop, and atomically combines the private sums into the master variable at loop exit
- **c)** It turns off compiler optimizations so numbers add up slowly
- **d)** It writes each addition operation to a temporary text file on disk

> **TIP**
> **Correct Answer: [b]** — It allocates a private local copy of `total_sum` initialized to 0 for each thread, accumulates locally without synchronization during the loop, and atomically combines the private sums into the master variable at loop exit
> **Technical Explanation:** The `reduction(op:var)` clause eliminates data races and lock contention by generating thread-private accumulators initialized to the operator's identity element (0 for addition). At the end of the parallel construct, the runtime reduces all private copies into the shared variable safely.

Question 08 [Module 2: OpenMP for Multithreading | Medium | Memory Scoping]

Topic: Variable Scoping: Private vs Shared
Question: What is the critical bug in the following OpenMP loop: `#pragma omp parallel for shared(idx) for (idx=0; idx<N; idx++) { process(idx); }`?

- **a)** The loop index `idx` is declared `shared`, causing all threads to concurrently overwrite the loop counter and corrupt iteration bounds
- **b)** The `process()` function cannot accept integer arguments
- **c)** OpenMP loops must always count downwards from N to 0
- **d)** C++ prohibits using `for` loops inside pragmas

> **TIP**
> **Correct Answer: [a]** — The loop index `idx` is declared `shared`, causing all threads to concurrently overwrite the loop counter and corrupt iteration bounds
> **Technical Explanation:** Loop iteration variables in parallel loops must ALWAYS be private to each thread. Sharing `idx` means multiple threads simultaneously read and increment the exact same memory location, leading to race conditions, skipped iterations, and unpredictable crashes.

Question 09 [Module 2: OpenMP for Multithreading | Hard | Load Balancing]

Topic: OpenMP Work-Sharing Schedules
Question: You are parallelizing an algorithm where iteration `i` takes $O(i^2)$ time (each subsequent iteration takes exponentially more computation). Which OpenMP loop scheduling clause provides the best load balancing across threads?

- **a)** `schedule(static, 100)`
- **b)** `schedule(dynamic, 1)` or `schedule(guided)`
- **c)** `schedule(none)`
- **d)** `schedule(random)`

> **TIP**
> **Correct Answer: [b]** — `schedule(dynamic, 1)` or `schedule(guided)`
> **Technical Explanation:** With `static` scheduling, chunks are assigned before loop execution; threads assigned later chunks with $O(i^2)$ work would execute long after earlier threads finish idle. `dynamic` scheduling uses a shared work queue where threads grab small chunks upon completing work, maintaining near-perfect load balance.

Question 10 [Module 2: OpenMP for Multithreading | Easy | Runtime Configuration]

Topic: Controlling Thread Count in OpenMP
Question: Which environment variable is read by the OpenMP runtime at program startup to set the default number of worker threads?

- **a)** `NUM_CPUS`
- **b)** `OMP_NUM_THREADS`
- **c)** `INTEL_MAX_THREADS`
- **d)** `THREAD_POOL_SIZE`

> **TIP**
> **Correct Answer: [b]** — `OMP_NUM_THREADS`
> **Technical Explanation:** `OMP_NUM_THREADS` is the official standard OpenMP environment variable (e.g., `export OMP_NUM_THREADS=16`) that specifies the default number of threads to use in parallel regions unless overridden in code.

Question 11 [Module 3: MPI for Distributed Systems | Easy | MPI Fundamentals]

Topic: MPI Communicators & `MPI_COMM_WORLD`
Question: In the Message Passing Interface (MPI) standard, what is the role of `MPI_COMM_WORLD`?

- **a)** It is an internet web browser built into MPI
- **b)** It is the default global communicator containing all processes launched when `mpirun` or `mpiexec` was invoked
- **c)** It is the root IP address of the supercomputer login node
- **d)** It is a configuration file on the Linux hard drive

> **TIP**
> **Correct Answer: [b]** — It is the default global communicator containing all processes launched when `mpirun` or `mpiexec` was invoked
> **Technical Explanation:** `MPI_COMM_WORLD` is the predefined communicator created upon `MPI_Init()`. It encompasses the set of all processes launched together across the cluster, providing a group context inside which ranks are uniquely indexed from 0 to N-1.

Question 12 [Module 3: MPI for Distributed Systems | Easy | MPI API]

Topic: Process Identification: Rank vs Size
Question: Which two fundamental MPI functions return the unique ID of the calling process and the total number of processes in the communicator respectively?

- **a)** `MPI_Get_id()` and `MPI_Get_total()`

- **b)** `MPI_Comm_rank()` and `MPI_Comm_size()`
- **c)** `MPI_Whoami()` and `MPI_Count()`
- **d)** `MPI_Process_rank()` and `MPI_Cluster_len()`

> **TIP**
> **Correct Answer: [b]** — `MPI_Comm_rank()` and `MPI_Comm_size()`
> **Technical Explanation:** `MPI_Comm_rank(comm, &rank)` returns the caller's unique integer ID (0 to size-1). `MPI_Comm_size(comm, &size)` returns the total count of processes in the communicator.

Question 13 [Module 3: MPI for Distributed Systems | Medium | MPI Communication]

Topic: Point-to-Point Message Envelope
Question: What 6 parameters compose the standard message envelope required by `MPI_Send(buf, count, datatype, dest, tag, comm)`?

- **a)** URL, port, username, password, token, encryption_key
- **b)** Memory buffer pointer, number of elements, MPI datatype, destination rank, message identification tag, and communicator
- **c)** Filename, line number, file size, permissions, owner, group
- **d)** CPU frequency, voltage, fan speed, core ID, socket ID, NUMA node

> **TIP**
> **Correct Answer: [b]** — Memory buffer pointer, number of elements, MPI datatype, destination rank, message identification tag, and communicator
> **Technical Explanation:** A standard MPI message envelope specifies: buffer address, element count, datatype (`MPI_INT`, `MPI_DOUBLE`), destination rank, message tag (for matching/filtering), and the communicator.

Question 14 [Module 3: MPI for Distributed Systems | Medium | MPI Collectives]

Topic: Collective Operations: Broadcast vs Scatter
Question: What is the key functional difference between `MPI_Bcast` and `MPI_Scatter` in distributed cluster computing?

- **a)** `MPI_Bcast` sends data only to Rank 0; `MPI_Scatter` sends data to all ranks
- **b)** `MPI_Bcast` copies the exact same data buffer identically to every process in the communicator; `MPI_Scatter` divides an array into equal segments and sends a distinct segment to each rank
- **c)** `MPI_Scatter` can only run on GPUs; `MPI_Bcast` only runs on CPUs
- **d)** `MPI_Bcast` deletes the data after sending

> **TIP**
> **Correct Answer: [b]** — `MPI_Bcast` copies the exact same data buffer identically to every process in the communicator; `MPI_Scatter` divides an array into equal segments and sends a distinct segment to each rank
> **Technical Explanation:** In `MPI_Bcast`, the root process broadcasts the entire buffer identically to all ranks. In `MPI_Scatter`, the root process takes an array of size $N \times P$ and distributes chunk i of size N uniquely to process rank i.

Question 15 [Module 3: MPI for Distributed Systems | Medium | MPI Collectives]

Topic: Global Reduction: `MPI_Reduce` vs `MPI_Allreduce`
Question: When computing the global loss across distributed nodes in deep learning or parallel ML, why is `MPI_Allreduce` used instead of `MPI_Reduce`?

- **a)** `MPI_Allreduce` performs the mathematical reduction across all processes and distributes the final reduced result to EVERY process; `MPI_Reduce` only leaves the final result on the designated root rank
- **b)** `MPI_Allreduce` operates without network cables
- **c)** `MPI_Reduce` is deprecated and deleted from the MPI standard
- **d)** `MPI_Allreduce` only supports integer data

> **TIP**
> **Correct Answer: [a]** — `MPI_Allreduce` performs the mathematical reduction across all processes and distributes the final reduced result to EVERY process; `MPI_Reduce` only leaves the final result on the designated root rank
> **Technical Explanation:** `MPI_Reduce` combines values and stores the final answer exclusively on the root rank. `MPI_Allreduce` reduces values and broadcasts the result to every participating process, which is necessary so every worker node can update its local parameters.

Question 16 [Module 4: Data Decomposition & Synchronization | Medium | Parallel Algorithms]

Topic: Domain Decomposition Strategies
Question: In scientific grid simulations and 2D image processing, why is 2D domain decomposition (grid checkerboard) superior to 1D domain decomposition (slab slicing) at high processor counts?

- **a)** 2D decomposition reduces the surface-to-volume communication ratio, requiring significantly less boundary data to be exchanged over the network as core count increases
- **b)** 1D decomposition cannot handle floating-point numbers
- **c)** 2D decomposition eliminates the need for RAM
- **d)** 1D decomposition only works on Intel processors

> **TIP**
> **Correct Answer: [a]** — 2D decomposition reduces the surface-to-volume communication ratio, requiring significantly less boundary data to be exchanged over the network as core count increases
> **Technical Explanation:** Surface-to-volume ratio governs communication overhead. For a domain of size $N \times N$ divided among P processors: 1D slab decomposition exchanges $2N$ boundary elements per step; 2D block decomposition exchanges $4(N/\sqrt{P})$, which scales down communication volume dramatically as P grows.

Question 17 [Module 4: Data Decomposition & Synchronization | Hard | Grid Synchronization]

Topic: Halo / Ghost Cell Boundary Exchange
Question: What is the primary purpose of 'Halo' or 'Ghost' cells in distributed domain decomposition stencil computations?

- **a)** To hide secret passwords from other users on the supercomputer
- **b)** To replicate boundary memory layers from neighboring subdomains locally, enabling each process to compute stencil updates independently without querying remote memory during the compute phase
- **c)** To convert single-precision floats into double-precision floats automatically

- **d)** To store backup copies of the operating system kernel

> **TIP**
> **Correct Answer: [b]** — To replicate boundary memory layers from neighboring subdomains locally, enabling each process to compute stencil updates independently without querying remote memory during the compute phase
> **Technical Explanation:** Halo (ghost) cells are extra memory padding allocated around a local subdomain grid. Prior to computation, processes exchange boundary borders with immediate physical neighbors. This allows the inner compute loop to access all stencil neighbors locally from cache without remote network fetches.

Question 18 [Module 4: Data Decomposition & Synchronization | Easy | Concurrency Hazards]

Topic: Data Races in Shared Memory
Question: Under what exact condition does a 'Data Race' occur in a multi-threaded OpenMP program?

- **a)** When two threads execute on different CPU sockets at different clock speeds
- **b)** When two or more concurrent threads access the exact same memory location without synchronization, and at least one of the accesses is a write operation
- **c)** When a thread takes longer than 1 millisecond to complete a loop
- **d)** When the compiler uses `-O3` optimization

> **TIP**
> **Correct Answer: [b]** — When two or more concurrent threads access the exact same memory location without synchronization, and at least one of the accesses is a write operation
> **Technical Explanation:** A data race formally occurs when: (1) two or more instructions in different threads access the same memory location concurrently, (2) at least one of these accesses is a write, and (3) there is no synchronization (mutex, atomic, barrier) ordering the accesses.

Question 19 [Module 4: Data Decomposition & Synchronization | Medium | Distributed Hazards]

Topic: Deadlock in Distributed MPI
Question: Consider two MPI processes: Rank 0 executes `MPI_Recv(from Rank 1)` then `MPI_Send(to Rank 1)`. Simultaneously, Rank 1 executes `MPI_Recv(from Rank 0)` then `MPI_Send(to Rank 0)`. What happens?

- **a)** The data is transferred successfully in 1 millisecond
- **b)** Both processes enter an indefinite Deadlock, each blocked waiting for the other to send a message that cannot be sent until the receive completes
- **c)** MPI automatically terminates Rank 1 and restarts the job
- **d)** The operating system converts the blocking calls into non-blocking calls automatically

> **TIP**
> **Correct Answer: [b]** — Both processes enter an indefinite Deadlock, each blocked waiting for the other to send a message that cannot be sent until the receive completes
> **Technical Explanation:** This is a classic circular wait deadlock. Because standard blocking `MPI_Recv` will not return until matching data arrives, neither process can reach its subsequent `MPI_Send`, leaving both processes hung indefinitely. The solution is using `MPI_Sendrecv` or non-blocking `MPI_Irecv`.

Question 20 [Module 4: Data Decomposition & Synchronization | Hard | Synchronization Primitives]

Topic: Synchronization: `#pragma omp critical` vs `atomic`
Question: Why is `#pragma omp atomic` significantly faster than `#pragma omp critical` for updating a shared counter (`counter++`) in OpenMP?

- **a)** `#pragma omp atomic` turns off cache coherency across cores
- **b)** `#pragma omp atomic` leverages dedicated hardware CPU atomic instructions (e.g., `LOCK INC` or `CMPXCHG`), whereas `#pragma omp critical` establishes a heavyweight software mutex lock across all threads
- **c)** `#pragma omp critical` only works in C, while `atomic` works in C++
- **d)** `#pragma omp atomic` skips writing to physical RAM

> **TIP**
> **Correct Answer: [b]** — `#pragma omp atomic` leverages dedicated hardware CPU atomic instructions (e.g., `LOCK INC` or `CMPXCHG`), whereas `#pragma omp critical` establishes a heavyweight software mutex lock across all threads
> **Technical Explanation:** `atomic` maps directly to low-level hardware instructions supported natively by the CPU architecture (such as x86 atomic fetch-and-add). `critical` invokes a generalized software lock/mutex runtime wrapper that introduces high latency and serializes all threads.

Question 21 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Easy | oneAPI Ecosystem]

Topic: Intel oneDAL Architecture
Question: What is the primary role of the Intel oneAPI Data Analytics Library (oneDAL) within the Intel AI Analytics Toolkit?

- **a)** To scrape websites for financial stock prices
- **b)** To provide highly optimized, hardware-accelerated algorithmic kernels for all data analysis stages (preprocessing, transformation, classical ML training and inference) leveraging AVX-512 and AMX on Intel architectures
- **c)** To convert Python code into Java bytecode
- **d)** To manage physical hard drive partitions

> **TIP**
> **Correct Answer: [b]** — To provide highly optimized, hardware-accelerated algorithmic kernels for all data analysis stages (preprocessing, transformation, classical ML training and inference) leveraging AVX-512 and AMX on Intel architectures
> **Technical Explanation:** Intel oneDAL delivers vectorized, multi-threaded algorithms for classical machine learning and data analytics. It dynamically detects host CPU vector extensions (AVX-512, AMX) to maximize compute and memory throughput for algorithms like PCA, KMeans, and SVM.

Question 22 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Easy | Applied ML Acceleration]

Topic: Drop-in Patching via `patch_sklearn()`
Question: How can an existing Python machine learning project using Scikit-Learn achieve Intel oneDAL acceleration without modifying model training or prediction code?

- **a)** By rewriting all Python models in CUDA C++
- **b)** By adding `from sklearnex import patch_sklearn; patch_sklearn()` at the top of the Python script before importing any Scikit-learn modules

- **c)** By running `export FORCE_INTEL_GPU=1` in the Linux terminal
- **d)** By renaming all `.py` files to `.cpp`

> **TIP**
> **Correct Answer: [b]** — By adding `from sklearnex import patch_sklearn; patch_sklearn()` at the top of the Python script before importing any Scikit-learn modules
> **Technical Explanation:** `scikit-learn-intelex` (sklearnex) provides transparent drop-in patching. Calling `patch_sklearn()` dynamically monkey-patches Scikit-learn classes (such as `KMeans`, `SVC`, `RandomForestClassifier`), rerouting execution to oneDAL's native C++ kernels while maintaining 100% API compatibility.

Question 23 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Algorithmic Acceleration]

Topic: K-Means Clustering Acceleration Mechanisms
Question: Why does Intel Extension for Scikit-learn typically deliver 10x to 40x speedups on K-Means clustering benchmarks compared to standard Scikit-learn?

- **a)** It uses random guessing to pick cluster centers in zero time
- **b)** It utilizes vectorized AVX-512 distance computation kernels, cache-friendly data layouts, and parallelized Lloyd's algorithmic updates optimized for Intel Xeon memory hierarchies
- **c)** It limits clustering to a maximum of 2 clusters
- **d)** It downsamples the dataset by deleting 90% of the rows

> **TIP**
> **Correct Answer: [b]** — It utilizes vectorized AVX-512 distance computation kernels, cache-friendly data layouts, and parallelized Lloyd's algorithmic updates optimized for Intel Xeon memory hierarchies
> **Technical Explanation:** Standard Scikit-learn relies on generic Cython loops that bottleneck on memory transfers and scalar math. oneDAL reformulates distance calculations using vectorized FMA instructions (AVX-512), tiles memory to fit into L2/L3 caches, and balances centroid recalculation across all available CPU threads.

Question 24 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Algorithmic Acceleration]

Topic: Support Vector Machine (SVM) Acceleration
Question: Which computational bottleneck in classical Support Vector Machines (SVC) is accelerated by Intel oneDAL on Intel Xeon CPUs?

- **a)** HTML parsing and data visualization
- **b)** Sequential Minimal Optimization (SMO) kernel matrix evaluations and working set selection
- **c)** Audio decoding and speech synthesis
- **d)** Hard disk defragmentation

> **TIP**
>
> **Correct Answer: [b]** — Sequential Minimal Optimization (SMO) kernel matrix evaluations and working set selection
> **Technical Explanation:** SVM training is bottlenecked by quadratic kernel evaluations ($O(N^2)$) during SMO optimization. Intel oneDAL uses parallelized matrix caches and vectorized kernel evaluations (RBF, linear, polynomial) to speed up SVM training by up to 100x over stock libsvm.

Question 25 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Easy | Applied ML Acceleration]

Topic: Restoring Baseline Scikit-Learn Behavior
Question: If a developer needs to unpatch Scikit-learn to compare empirical benchmark results against stock Scikit-learn in the same Python session, what function should they invoke?

- **a)** `sklearnex.reset_system()`
- **b)** `sklearnex.unpatch_sklearn()`
- **c)** `scipy.restore_defaults()`
- **d)** `sys.exit_intel()`

> **TIP**
>
> **Correct Answer: [b]** — `sklearnex.unpatch_sklearn()`
> **Technical Explanation:** `sklearnex.unpatch_sklearn()` reverts all Scikit-learn module references back to their original stock implementations, allowing seamless A/B performance comparison in a single benchmark script.
